

IHK UML & Klassen – ترفندهای امتحان و

مثال‌های تصویری

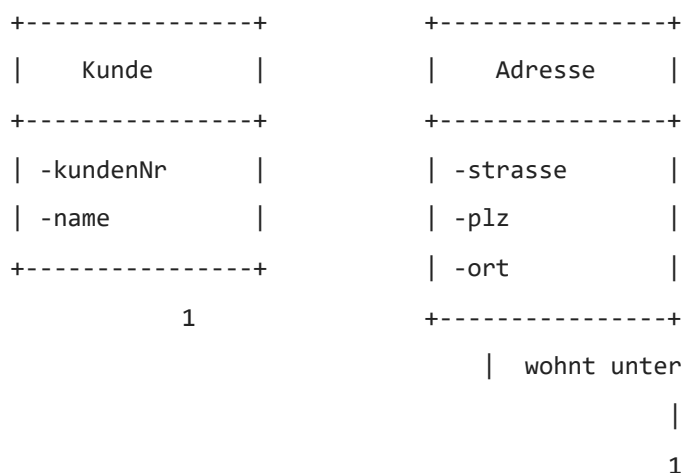
این راهنما به تو کمک می‌کند دیاگرام‌های کلاس UML را در امتحان IHK سریع بخوانی، نوع رابطه‌ها را تشخیص دهی و از روی آن‌ها کلاس‌ها و متدهای درست را طراحی کنی.

۱. انواع رابطه در UML – Überblick

- Assoziation – ارتباط ساده بین دو کلاس (می‌شناسند هم‌دیگر)
- Aggregation – رابطه «داراست» ولی Teil می‌تواند مستقل وجود داشته باشد
- Komposition – رابطه «داراست» قوی؛ Teil بدون Ganzes وجود ندارد
- Vererbung – ارث‌بری، یک کلاس زیرکلاس دیگری است

۲. Assoziation – رابطه ساده بین دو کلاس

مثال: Kunde – Adresse

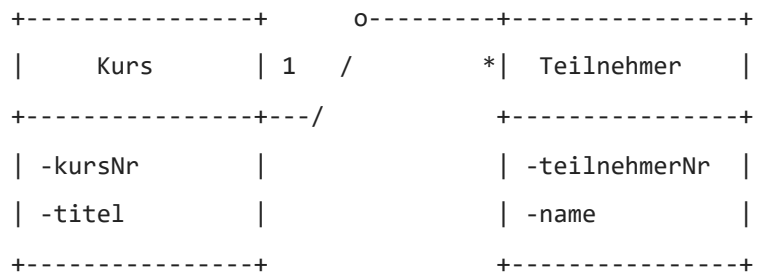


در این مثال، هر مشتری یک آدرس دارد و آدرس فقط به همان مشتری مربوط است (در این مدل ساده). این یک Assoziation است، چون فقط نشان می‌دهد دو کلاس با هم مرتبط‌اند.

- فلش یا نام رابطه (مثلاً wohnt unter) فقط معنی رابطه را توضیح می‌دهد.
- Multiplicity (مثل ۱ و ۱) نشان می‌دهد هر سمت چند شیء می‌تواند داشته باشد.

۲. Aggregation – Teil kann eigenständig existieren

مثال: Kurs – Teilnehmer

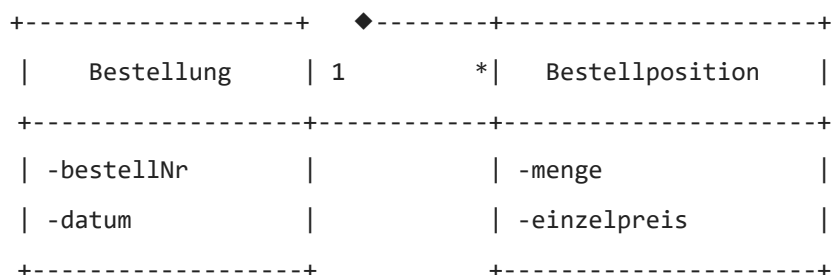


دایره توخالی در سمت کلاس Kurs نشان می‌دهد که این یک Aggregation است.

- یک Kurs شامل چند Teilnehmer است (۱ به *).
- Teilnehmer می‌تواند بدون Kurs هم وجود داشته باشد (مثلاً در سیستم به‌طور کلی ثبت شده است).
- اگر یک Kurs حذف شود، ممکن است Teilnehmer ها در سیستم باقی بمانند.

۴. Komposition – Teil gehört fest zum Ganzen

مثال: Bestellung – Bestellposition



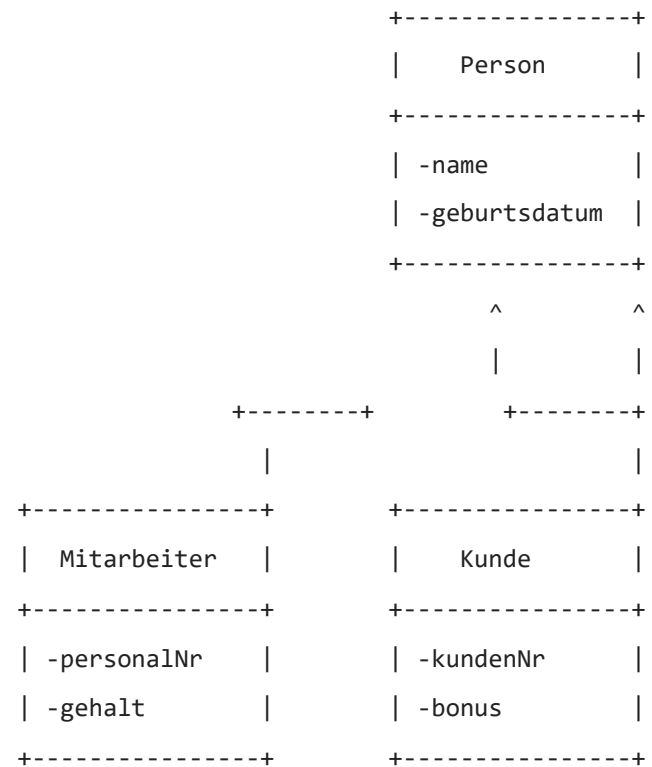
لوزی پر (◆) در سمت Bestellung نشان‌دهنده Komposition است.

- هر Bestellung از چند Bestellposition تشکیل شده است.

- یک Bestellposition بدون Bestellung معنی ندارد.
- اگر Bestellung حذف شود، تمام Bestellposition ها نیز حذف می‌شوند.

۵. Vererbung – ارث‌بری بین کلاس‌ها

مثال: Person – Mitarbeiter – Kunde



فلش مثلثی رو به بالا نشان می‌دهد که کلاس‌های Mitarbeiter و Kunde از کلاس Person ارث‌بری می‌کنند.

- فیلدهای مشترک مانند نام و تاریخ تولد در کلاس پایه Person آمده‌اند.
- هر زیرکلاس فیلدهای مخصوص خود را دارد.

۶. Multiplicity – تعداد اشیاء در هر سمت رابطه

علامت	معنی
1	دقیقاً یک شیء

اختیاری، حداکثر یک شیء	1..0
صفر تا تعداد بی‌نهایت	*
حداقل یک شیء، حداکثر بی‌نهایت	*..1

مثال مهم IHK: $Kunde\ 1..* \text{ --- } 0..* \text{ Bestellung}$ یعنی هر Kunde می‌تواند چند Bestellung داشته باشد و هر Bestellung دقیقاً به یک Kunde تعلق دارد (غالباً $*..1$ zu 1 modelliert).

۷. تبدیل UML به ساختار کلاس

مثال: Kunde – Rechnung

+-----+ 1 +-----+	
Kunde ----- Rechnung	
+-----+ * +-----+	
-kundenNr	-rechnungsNr
-name	-betrag
+-----+	-datum
	+-----+

این دیاگرام نشان می‌دهد که یک Kunde می‌تواند چند Rechnung داشته باشد (۱ به *).

کلاس‌ها در شبه‌کد

```

klasse Kunde
    attribute
        kundenNr: Integer
        name: String
        rechnungen: Array von Rechnung // 1..* Beziehung

    methoden
        konstruktor(kundenNr: Integer, name: String)
        fuegeRechnungHinz(r: Rechnung)
        gibGesamtUmsatz(): Integer
    end klasse

klasse Rechnung

```

```
attribute
rechnungsNr: Integer
betrag: Integer
datum: String
methoden
konstruktor(rechnungsNr: Integer, betrag: Integer, datum: String)
end klasse
```

نکته ترفندی:

- کلاسی که در Multiplicity سمت ۱ است (Kunde) معمولاً مالک لیست سمت * می‌شود.
- بنابراین آرایه یا لیست rechnungen باید در کلاس Kunde قرار گیرد.

۸. کلاس‌های مدیریتی (Verwaltung / Manager)

در بسیاری از امتحانات، کلاس‌هایی مثل KundenVerwaltung یا KursManager وجود دارند. این کلاس‌ها معمولاً:

- یک لیست از اشیاء دیگر را نگه می‌دارند (مثلاً kunden: Array von Kunde).
- متدهایی مانند hinzufuegen , loeschen , sucheNach... دارند.

مثال: KundenVerwaltung

```
klasse KundenVerwaltung
attribute
kundenListe: Array von Kunde

methoden
fuegeKundeHinzu(k: Kunde)
sucheKundeNachNummer(nr: Integer): Kunde
entferneKunde(nr: Integer)
end klasse
```

اینجا خود کلاس Kunde ساده می‌ماند و منطق جستجو و مدیریت در کلاس Verwaltungs پیاده‌سازی می‌شود.

۹. Getter / Setter / Konstruktor – الگوی استاندارد

در IHK معمولاً نیازی به پیاده‌سازی کامل Getter/Setter نیست، اما باید بدان‌ی الگو چیست.

```

    klasse Kunde
        attribute
            kundenNr: Integer
            name: String

        methoden
            konstruktor(kundenNr: Integer, name: String)
                this.kundenNr ← kundenNr
                this.name ← name
            end konstruktor

            getKundenNr(): Integer
                rückgabe kundenNr
            end methode

            setName(neuerName: String)
                name ← neuerName
            end methode
        end klasse
```

۱۰. مثال ترکیبی: Kurs – Teilnehmer – KursVerwaltung

```

+-----+      o-----+-----+
|      Kurs      | 1  /      * | Teilnehmer  |
+-----+-----+      +-----+
| -kursNr        |          | -teilnehmerNr |
| -titel         |          | -name         |
+-----+-----+      +-----+

+-----+
| KursVerwaltung |
+-----+
| -kurse: Array<Kurs> |
+-----+
| +fuegeKursHinzu(k) |
```

در این مدل:

- Aggregation بین Kurs و Teilnehmer: Teilnehmer می‌تواند مستقل نیز وجود داشته باشد.
- Kursverwaltung مدیریت لیست Kurs را انجام می‌دهد.

۱۱. نکات طلایی امتحان UML

- سمت ۱ معمولاً صاحب لیست سمت * است.
- Aggregation: Teil بدون Ganzes هم می‌تواند وجود داشته باشد.
- Komposition: Teil بدون Ganzes معنی ندارد.
- در Vererbung، ویژگی‌های مشترک در کلاس پایه قرار می‌گیرند.
- اگر کلاس اسم Verwaltung یا Manager دارد، کار آن مدیریت لیست است نه نگاه‌داشتن جزئیات منطقی هر شیء.

۱۲. Antwortsätze auf Deutsch (für Theoriefragen)

در این بخش، چند جمله آماده آلمانی آورده شده است که می‌توانی در پاسخ به سؤالات نظری UML استفاده کنی. فقط آلمانی هستند، بدون ترجمه.

Allgemeine UML-Fragen

- Ein Klassendiagramm beschreibt die statische Struktur eines Softwaresystems mit,
".Klassen, Attributen, Methoden und Beziehungen"
- Assoziationen stellen logische Beziehungen zwischen zwei Klassen dar, zum Beispiel,
".Kunde hat Rechnungen"
- Die Multiplizität gibt an, wie viele Objekte einer Klasse mit einem Objekt der,
".anderen Klasse verbunden sein können"

Aggregation und Komposition

Eine Aggregation ist eine schwache Form der 'hat-ein'-Beziehung. Das Teilobjekt,, •

„.kann auch ohne das Ganze existieren

Eine Komposition ist eine starke 'besteht-aus'-Beziehung. Das Teilobjekt kann ohne,, •

„.das Ganze nicht sinnvoll existieren

Bei einer Komposition werden Teilobjekte in der Regel mit dem Ganzen erzeugt und,, •

„.mit ihm wieder gelöscht

Vererbung

Vererbung beschreibt eine 'ist-ein'-Beziehung zwischen einer Basisklasse und einer,, •

„.abgeleiteten Klasse

Gemeinsame Attribute und Methoden werden in der Oberklasse definiert und von,, •

„.den Unterklassen geerbt

Vererbung fördert Wiederverwendung von Code und eine klare Strukturierung des,, •

„.Modells

Beziehung Kunde – Rechnung

Ein Kunde kann mehrere Rechnungen besitzen, daher modelliert man eine 1-zu-n,, •

„.Beziehung zwischen Kunde und Rechnung

„.In der Klasse Kunde wird häufig eine Liste von Rechnungen als Attribut geführt,, •

„.Die Rechnung kennt genau einen Kunden, dem sie zugeordnet ist,, •

Kurs – Teilnehmer

Ein Kurs kann viele Teilnehmer haben, ein Teilnehmer kann an mehreren Kursen,, •

„.teilnehmen. Dies ist eine n-zu-m-Beziehung

n-zu-m-Beziehungen werden in relationalen Modellen häufig über eine,, •

„.Zwischentabelle aufgelöst

„.Im Objektmodell kann die Kursklasse eine Liste von Teilnehmern verwalten,, •

Verwaltungsklassen / Manager

Verwaltungsklassen wie 'KundenVerwaltung' kapseln Operationen zum Hinzufügen,, •

„.Suchen und Entfernen von Objekten

Sie sorgen für eine klare Trennung zwischen den Fachobjekten und der Steuerlogik,, •
".des Systems

پایان راهنمای UML. این فایل را می‌توانی در مرورگر باز کنی، چاپ بگیری یا به PDF تبدیل کنی و
به‌عنوان مرجع سریع برای تمرین سؤال‌های UML در IHK استفاده کنی.